Nicolás Andrés Cano Leal

Backend Engineer → SRE | Python · Django · Kubernetes · Terraform · CI/CD

Rancagua, Chile (remoto) · nicolas.cano.leal@gmail.com · +56 9 9411 3328 · github.com/NicolasAndresCL
· linkedin.com/in/nicolas-andres-cano-leal · nicolasandrescl.pythonanywhere.com

PERFIL

Ingeniero backend en Python que construye servicios pensando en cómo se operan, no solo en cómo se escriben. Soy el único desarrollador de un equipo de operaciones en PedidosYa, donde reemplazo procesos manuales por servicios Python testeados que corren desatendidos contra BigQuery, Google Sheets y sistemas internos. Trabajo con Django y DRF, y llevo cada proyecto hasta producción con CI/CD, contenedores e infraestructura como código. Mi objetivo es Site Reliability Engineering: sistemas que escalan, se observan y se recuperan solos.

EXPERIENCIA

PedidosYa — Live Performance Agent (LiveOps / BizOps)

mar. 2026 – presente

Rancagua, Chile (remoto) · Único desarrollador del equipo de operaciones

Bot de automatización de operaciones en tiempo real

- Evolucioné el bot interno a través de 10 versiones (v4.0 → v5.0), con 481 tests automatizados y 144 commits bajo convenciones profesionales.
- Orquesté ~10 motores concurrentes en hilos (alertas, reporting, watchdogs) y cerré una condición de carrera *check-then-act* entre ellos con un cooldown de reserva atómica.
- Reduje las llamadas a Google Sheets de 21 round-trips a 1 con un singleton con caché TTL (~16 min → <1 min) y bajé el cuello de botella global de ~37 min a ~10 min.
- Diseñé en capas: lógica pura separada de Selenium, red y BigQuery, testeable sin abrir un navegador.

Recolector de dossiers de incidencias

- Construí un backend Django/DRF con UI en NiceGUI que reúne la evidencia de cada incidencia desde BigQuery y reemplaza el llenado manual de planillas y correos.
- Una auditoría medida detectó que la fuente principal había caído al 0,11 % de cobertura; migré el timeline a una tabla con 100 % de cobertura y 13 meses de retención. 600+ tests.

Sincronización de participantes de órdenes canceladas

- Servicio que lee un stream de eventos en BigQuery (solo lectura) y publica de forma idempotente en Google Sheets, sin pisar lo que anotan los analistas.
- Arquitectura de puertos y adaptadores: cambiar el destino de BigQuery a Sheets costó configuración, no código de dominio. Decisiones registradas en ADRs con diagramas UML.
- Operando sobre 60.000+ filas: diagnostiqué 923 celdas corruptas por proporción estadística y migré el esquema de la hoja en vivo sin perder datos. 312 tests, 80 % de cobertura.

Python · Django/DRF · NiceGUI · BigQuery · Google Sheets API · Selenium · Pandas · pytest · GitHub Actions

PROYECTOS

All in Django — backend con infraestructura completa como código

repo

- API Django + DRF para cinco dominios operativos, con UI cliente en NiceGUI que solo habla HTTP. 229 tests, ~83 % de cobertura exigida en CI.
- Contrato OpenAPI (drf-spectacular) verificado con una colección Postman de 79 peticiones y 183 aserciones que Newman corre en CI contra un stack efímero; un test falla si un endpoint queda sin probar.
- CI de 6 jobs (GitHub Actions) y CD con Jenkins: release v1.0.0 publicada a GHCR y desplegada con respaldo previo y rollback automático.
- Terraform (EC2 + RDS PostgreSQL) probado sin cuenta de AWS con `terraform test` y proveedores simulados; la prueba destapó tres fallos latentes antes del primer despliegue. Helm para Kubernetes.

Portafolio personal — Django 5.2 LTS + React (en producción) repo

- Migré producción de SQLite a PostgreSQL 16 gestionado (Neon) sin pérdida de datos, tras diagnosticar que el Postgres del hosting (v12) era incompatible con Django 5.2.
- Configuración tipada con pydantic-settings, throttling, paginación, `/healthz/`, HTTPS endurecido por entorno. Type checking con mypy + django-stubs, 49 tests, ~91 % de cobertura.
- CI en GitHub Actions y CD con Jenkins en Docker (test gate → imagen → GHCR → deploy).

Infraestructura reproducible con Docker

- n8n-docker-local: n8n con versión fijada, volumen externo protegido, healthcheck real y CI que verifica que el servicio arranca.
- ultimate-linux-lab: Ubuntu 24.04 LTS con systemd real en un contenedor desechable, en reemplazo de una máquina virtual.

Otros proyectos

APIs con DRF y GraphQL (JWT, OpenAPI), e-commerce Django + React, CRUD con FastAPI + SQLAlchemy, refactor de Flask monolítico a API REST, dashboards en Streamlit y un modelo de regresión con scikit-learn. Catálogo completo en el portafolio.

HABILIDADES TÉCNICAS

Lenguajes	Python (avanzado), SQL, JavaScript, Java (en formación), Bash
Backend	Django, Django REST Framework, FastAPI, Flask, SQLAlchemy, pydantic, drf-spectacular (OpenAPI), GraphQL, JWT
Bases de datos	PostgreSQL (Neon, RDS), MySQL, SQLite, BigQuery
DevOps / IaC	Docker (multi-stage, no-root), Docker Compose, GitHub Actions, Jenkins, GHCR, Terraform, Helm, Kubernetes, Linux, AWS (EC2, RDS)
Calidad	pytest, coverage, mypy, Postman + Newman, pruebas de contrato, ADRs, UML
Automatización	Selenium, n8n, Google Sheets API, BeautifulSoup, Pandas, ETL idempotente
Interfaces	React, Vite, NiceGUI, Streamlit, Plotly
Herramientas	Git (conventional commits), desarrollo asistido por IA (Claude Code + MCP), VS Code

FORMACIÓN

- **Ingeniería en Informática** — IP Santo Tomás (en curso)
- **Técnico en Enfermería**, mención Instrumentista Quirúrgico — INACAP, 2014
- 2.500+ horas de estudio autodidacta en backend Python, arquitectura y DevOps

IDIOMAS

Español nativo · Inglés B1+ (lectura técnica fluida)

CERTIFICACIONES

- **Ultimate Docker** · **Ultimate Java** — Hola Mundo (2026)
- Ultimate Python · Ultimate Git + GitHub · Ultimate n8n — Hola Mundo (2026)
- SQL: Sin Fronteras — Hola Mundo (2025)
- Aplicaciones con LLMs (LangChain, Semantic Kernel, Prompt Flow) · GitHub Models — LinkedIn Learning (2026)
- Domina Python: PySpark · Python avanzado 2 — LinkedIn Learning (2026)
- Domina Python: Django REST · FastAPI · AWS para Profesionales IT — LinkedIn Learning (2025)
- Análisis de Datos con Python — Desafío Latam (2025)